Sub-Two-Second Multilingual Voice Agents for Outbound Lead Qualification: Sentence-Pipelined Streaming, Hysteresis Language Control and Database-Enforced Tenancy

[Author 1], [Author 2], [Author 3]
[Department], [Institution]
[City], India
{author1, author2, author3}@[institution].edu

***Abstract*—Outbound qualification of sales leads in India requires an agent that converses in English, Hindi and Tamil over narrowband telephony, tolerates code-mixing, and records every interaction under strict tenant separation. Cascaded speech pipelines meet these requirements poorly: response delay accumulates across recognition, generation and synthesis, and per-utterance language identification flips the conversation language on short or misrecognised utterances. This paper presents ASVA, a streaming voice agent that pipelines language-model output into sentence-level speech synthesis while a concurrent extractor fills eight qualification slots, controls the reply language with a hysteresis state machine, serves recurrent phrases from a content-addressed audio cache, and persists calls in a PostgreSQL schema whose tenant isolation and duration billing are enforced by row-level security and triggers. Over 421 logged turns from 52 pilot calls, the median time from end of speech to first agent audio was 1.57 s, and 77.6% of turns were answered within 2 s. Turns using server-side streaming recognition exhibited a 21% lower median delay than batch recognition, and turns served from the phrase cache a 39% lower median than synthesised turns. First-sentence generation accounted for 50.6% of the mean delay, identifying the language model as the dominant bottleneck; a controlled benchmark of three hosted models and three synthesis services quantifies the remaining headroom. Execution-based verification of the schema exposed three latent defects that would have rejected long calls, halted dispatch and nullified tenant claims. The results indicate that sub-two-second multilingual telephony agents are attainable with commodity cloud services when latency, language control and data integrity are treated as first-class design constraints.**

***Index Terms*—spoken dialogue systems, speech recognition, code-switching, large language models, multi-tenant databases, latency**

I. Introduction

Lead lists purchased by small brokers and business-to-business sellers decay quickly: a lead that is not contacted within minutes is frequently lost to a competitor. In India the contact must additionally happen in the lead's preferred language, which in a single metropolitan market such as Chennai spans English, Hindi and Tamil, often mixed within one sentence. Automating this first conversation with a voice agent is attractive, but three engineering constraints dominate its feasibility.

First, conversational turn-taking is fast. Cross-linguistic studies place the modal gap between turns near 200 ms [1], and long silences are attributed to inattention or to a machine [2]. A cascaded agent—automatic speech recognition (ASR), a large language model (LLM) and text-to-speech (TTS)—incurs a network round trip to each service per turn, over 8 kHz telephone audio that degrades recognition. Second, per-utterance language identification is unreliable precisely on the utterances most common in calls: one-word backchannels such as *haan* or *ok*, and English speech transcribed in an Indic script by a streaming recogniser. A policy of replying in the detected language consequently switches language mid-call. Third, a commercial deployment serves many customer organisations whose leads, transcripts and billing must be isolated; application-level filtering alone leaves isolation dependent on every query being written correctly.

Existing work addresses these problems separately: end-of-query detection and turn-taking models reduce endpointing delay [2], [3]; multilingual recognisers and Indic speech benchmarks improve recognition [4]–[6]; code-switching research characterises mixed-language speech [7]; and multi-tenant database designs address shared schemas [8]. Few published systems combine them into an operating telephony agent and report per-stage latency from real calls in Indian languages.

The principal contributions of this work are as follows:

- A sentence-pipelined streaming orchestration in which LLM tokens are segmented at sentence boundaries and each sentence is synthesised and transmitted while generation continues, with qualification-slot extraction executed concurrently off the critical path and a two-sentence output cap per turn.
- A hysteresis language state machine for code-mixed calls combining confidence gating, a two-utterance switch hysteresis, explicit switch requests, and script evidence treated as a vote rather than a decision.
- A two-tier phrase cache and endpointing policy for narrowband audio: content-addressed TTS reuse with a single voice pinned across languages, and server-side endpointing with merge and dangling-fragment holds over an amplitude-VAD fallback.

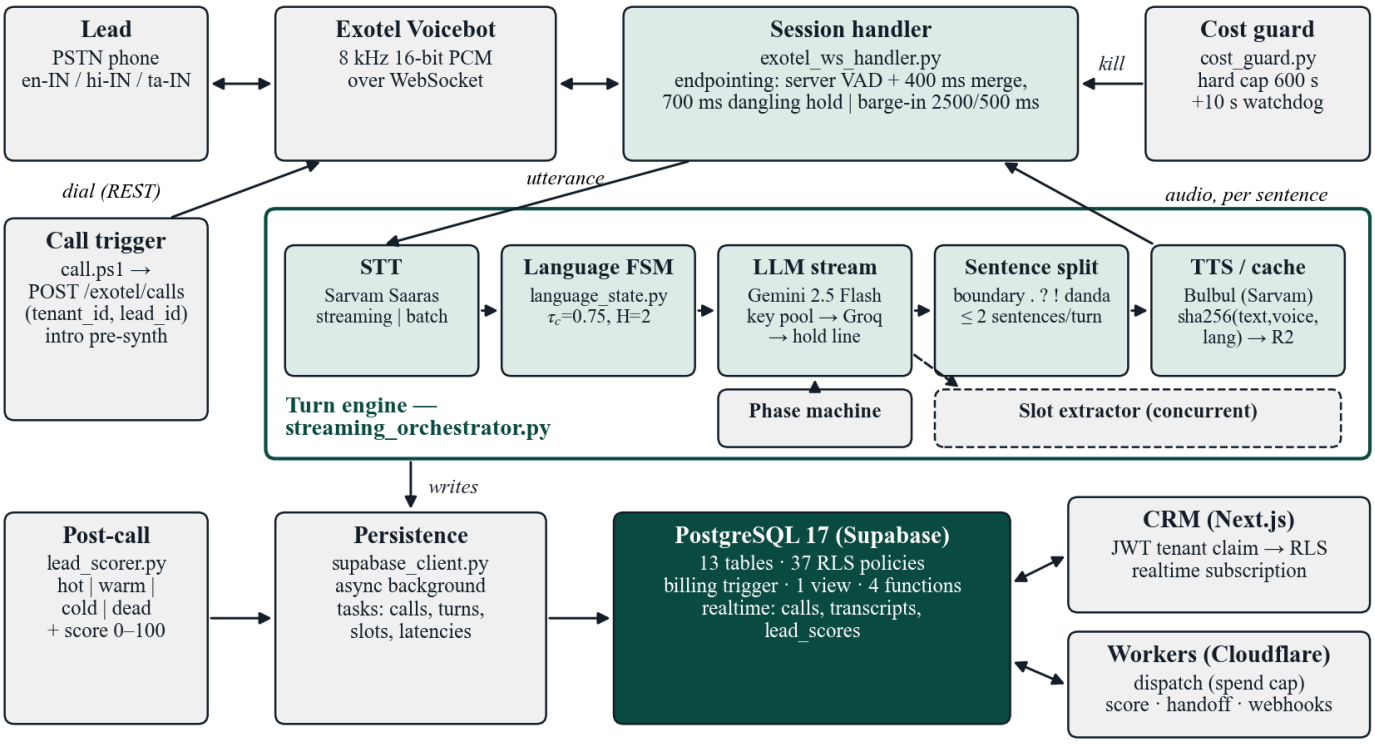


Fig. 1. System architecture. A call is requested through `POST /exotel/calls`; the session handler exchanges 8 kHz audio with the telephony provider and drives the turn engine, whose stages run left to right with the phase machine and a concurrent slot extractor beside the LLM stream. Writes are asynchronous; the CRM and workers access PostgreSQL under row-level security.

TABLE I
DESIGN PROPERTIES OF VOICE-AGENT PARADIGMS

Paradigm	Generation-to-TTS	Language control	Structured extraction	Tenant isolation
Sequential cascade (ASR, LLM, TTS)	sequential	follow utterance LID	post-call only	application level
Endpointing-optimised pipelines [2], [3]	partially streamed	not addressed	not addressed	not addressed
Multilingual ASR/TTS models [4], [5], [10]	n/a	model-level LID	n/a	n/a
This work	sentence-pipelined	hysteresis state machine	concurrent, confidence-merged	row-level security and triggers

- A database-enforced tenancy and billing design—row-level security keyed on a token claim, trigger-computed tiered billing and SQL spend guards—together with an execution-based verification that exposed three latent defects.
- An empirical characterisation from 421 logged turns and a controlled provider benchmark, including observational ablations of streaming recognition and caching.

Section II reviews related work. Section III formalises the architecture and its algorithms. Section IV describes the implementation and evaluation. Section V discusses scalability and limitations, and Section VI concludes.

II. RELATED WORK

A. Turn-Taking and Response Latency in Spoken Dialogue

Stivers et al. [1] measured turn transitions across ten languages and found a shared preference for minimal gaps. Skantze [2] surveys turn-taking models for conversational systems and identifies endpointing and response generation as the primary sources of machine delay. Shannon et al. [3] replace silence timeouts with a learned end-of-query detector in streaming ASR, reducing endpoint latency without increasing cut-offs. Dean and Barroso [9] show that tail latency governs perceived responsiveness in multi-service systems. The present work adopts server-side endpointing in the spirit of [3], adds conversational merge and dangling-fragment holds, attacks the generation component with sentence pipelining, and reports tail percentiles following [9].

B. Multilingual and Code-Switched Speech for Indian Languages

Weakly supervised multilingual ASR [4] and massively multilingual speech models [5] extend recognition to many Indian languages, while IndicSUPERB [6] benchmarks Indic speech tasks. Kumar et al. [10] build TTS for Indian languages at scale, and conditional variational TTS [11] underlies many modern neural voices. Sitaram et al. [7] survey code-switched speech processing and note that utterance-level language

identification is unreliable for short and mixed utterances. Rather than improving language identification itself, the present work treats its output as noisy evidence and controls the reply language with a hysteresis state machine.

C. LLM Agents, Lead Qualification and Multi-Tenant Data

General-purpose LLMs such as Gemini [12] enable open-domain dialogue but introduce hallucination risk [13], which motivates constrained outputs and structured extraction. Predictive lead scoring has been studied with conventional classifiers over CRM features [14]; here qualification is performed conversationally and scored from the transcript. Aulbach et al. [8] compare schema-mapping techniques for multi-tenant databases; the present design uses a shared schema with PostgreSQL row-level security [15] so that isolation is enforced by the database engine. Open-source frameworks such as Pipecat [16] provide pipeline primitives on which the implementation builds. Table I contrasts the design properties of these paradigms.

III. Proposed Architecture and Methodology

A. System Overview

The system comprises (i) a telephony layer that dials the lead and exchanges 8 kHz 16-bit PCM over a WebSocket; (ii) a session handler that performs endpointing, barge-in detection and call-context resolution; (iii) a turn engine that runs recognition, normalisation, language control, streaming generation and sentence synthesis; (iv) dialogue controllers, namely a language state machine and a monotone phase machine; (v) a concurrent slot extractor and a post-call scorer; and (vi) a PostgreSQL persistence layer with row-level security, accessed by the agent, by serverless workers (dispatch, scoring, hand-off) and by a CRM that receives realtime updates (Fig. 1).

B. Latency Model

For turn k, the perceived response delay is the time to first audio

$$\mathrm{TTFA}_k = \Delta_{\mathrm{EP}} + \Delta_{\mathrm{STT}} + \Delta_{\mathrm{pre}} + \Delta_{\mathrm{LLM},1} + \Delta_{\mathrm{TTS},1} + \Delta_{\mathrm{net}}, \quad (1)$$

where the terms denote endpointing, recognition, pre-processing, generation of the first sentence, synthesis of the first sentence and transmission. The instrumented quantity is

$$\hat{T}_k = \Delta_{\mathrm{STT}} + \Delta_{\mathrm{LLM},1} + \Delta_{\mathrm{TTS},1}. \quad (2)$$

A sequential cascade presents $\sum_{i=1}^{m}\left(\Delta_{\mathrm{LLM},i} + \Delta_{\mathrm{TTS},i}\right)$ for an m-sentence reply; sentence pipelining reduces the perceived component to $\Delta_{\mathrm{LLM},1} + \Delta_{\mathrm{TTS},1}$ because sentence $i \geq 2$ is produced during playback of sentence $i-1$. With a cap $m \leq 2$, turn wall time is bounded by two sentences of playback. When the recogniser has finalised the transcript at the endpoint, $\Delta_{\mathrm{STT}} \approx 0$.

C. Endpointing

In streaming mode the recogniser's voice-activity model emits a final segment; the turn is held for a grace window $g =$ 400 ms, extended while speech activity is reported, and a segment ending in a dangling fragment is held an additional h = 700 ms and merged with its continuation. In fallback mode, 100 ms frames with peak amplitude below 300 are silent and a turn closes after $s = 750$ ms of silence, with a minimum utterance of 400 ms and a maximum of 8 s. Barge-in flushes queued agent audio when input peak amplitude exceeds 2500 for 500 ms.

D. Language State Machine

Let $\mathscr{L} = \{\mathrm{en}, \mathrm{hi}, \mathrm{ta}\}$ and let an utterance be $u = (\lambda, c, w, \sigma, x)$: recogniser label, confidence, word count, detected script and text. The state is (ℓ, p, n): current language, pending language and pending count. With threshold $\tau_c = 0.75$ and hysteresis $H = 2$,

$$\ell' = \begin{cases} m, & x \text{ requests } m \neq \ell, \\ \sigma, & \sigma \notin \{\emptyset, \ell\},\ w \geq 2,\ \neg\mathrm{ack}(x),\ n' \geq H, \\ \lambda, & c \geq \tau_c,\ w \geq 3,\ \lambda \neq \ell,\ n' \geq H, \\ \ell, & \text{otherwise}, \end{cases} \quad (3)$$

where n' is the pending count after the update ($n' = n+1$ if the candidate equals p, otherwise $n' = 1$ and p is reset to the candidate). Code-mixed utterances never switch ℓ; they only inform the response hint. On a switch, a bridge phrase for the pair (ℓ, ℓ') is spoken. The machine trades a delay of at most $H-1$ turns for genuine, unrequested switches against robustness to single misrecognised or backchannel utterances.

E. Dialogue Phases, Slots and Scoring

The phase $\varphi(t)$ is monotone non-decreasing in elapsed time with boundaries at 8, 35, 70, 130 and 140 s (greeting, connect, discover, qualify, close), and an extension phase until 290 s admitted only if the buying confidence satisfies $\beta \geq 0.6$; the boundaries precede the billing boundaries of (6). Eight slots s carry values v_s and confidences κ_s. A concurrent extractor returns (v'_s, κ'_s), merged by

$$v_s \leftarrow v'_s \quad \text{iff} \quad v'_s \neq \emptyset \ \wedge\ \kappa'_s \geq \kappa_s. \quad (4)$$

The live temperature at turn k is

$$\theta_k = \begin{cases} \text{hot}, & \beta \geq 0.7 \ \vee\ (\pi \neq \emptyset \wedge \delta \leq 30), \\ \text{warm}, & \exists s: v_s \neq \emptyset \ \wedge\ \beta \geq 0.3, \\ \text{cold}, & k \geq 3, \\ \text{unknown}, & \text{otherwise}. \end{cases} \quad (5)$$

where π is the pain-point slot and δ the stated purchase timeline in days. After the call, an LLM classifies the full transcript and final slots into hot, warm, cold or dead with a score in $[0, 100]$; malformed output falls back to a deterministic rule.

F. Billing, Cost and Availability

Billed credits for a completed call of duration d seconds are

$$b(d) = \min\left(4, \lceil d / 150 \rceil\right), \quad (6)$$

computed identically in the agent and by a `BEFORE INSERT OR UPDATE` trigger. The per-call variable cost is

$$C = \frac{t_L}{60} r_{\mathrm{STT}} + \max\left(0, \frac{t_A}{60} r_{\mathrm{TTS}} - h s_c\right) + \frac{T}{60} r_{\mathrm{tel}} + n c_{\mathrm{LLM}} + c_0, \tag{7}$$

with lead and agent speech times t_L, t_A, duration T, n LLM turns and h cache hits; the constants are INR 3.00/min, INR 3.00/min, INR 0.10 per hit, INR 1.80/min, INR 0.015 per turn and INR 0.20. For a 180 s call with $t_L = t_A = 72$ s, $n = 30$ and $h = 0$, $C =$ INR 13.25. A dispatcher admits a call iff the SQL-computed spend for the current IST day plus the projection does not exceed the tenant cap. LLM availability uses per-call key rotation over a provider pool, fallback to an OpenAI-compatible endpoint on HTTP 429, and a localised hold utterance when all providers fail.

G. Tenant Isolation

For a relation R with column `tenant_id`, the policy predicate is $P_R(r) \equiv r.\mathrm{tenant} = \tau(J)$, where $\tau(J)$ reads the tenant claim of the request token J, injected at issuance by a login hook. A child relation R_c referencing R_p uses $P_{R_c}(r) \equiv \exists q \in R_p : q.\mathrm{id} = r.\mathrm{fk} \wedge q.\mathrm{tenant} = \tau(J)$. Inserts are guarded by the same predicates through `WITH CHECK`.

H. Turn Algorithm

Algorithm 1 summarises one streaming turn.

Algorithm 1 Streaming turn

Require: utterance u, call context X, sentence cap $m = 2$
Ensure: audio sentences streamed to the line; X updated

if u is audio **then**
  $x \leftarrow \mathrm{STT}(u)$
else
  $x \leftarrow u.\mathit{text}$ ▷ *pre-finalised by streaming STT*
end if
$x \leftarrow \textsc{NormaliseEntities}(x)$
$(\ell, \mathit{bridge}) \leftarrow \textsc{LanguageFSM}(x, \lambda, c)$ ▷ *Eq. (3)*
if $\mathit{bridge} \neq \emptyset$ **then** $\textsc{Emit}(\mathrm{TTS}(\mathit{bridge}, \ell))$ **end if**
$\varphi \leftarrow \textsc{Phase}(t, \beta)$
$E \leftarrow$ **async** $\textsc{ExtractSlots}(x)$ ▷ *off critical path*
$\mathit{buf} \leftarrow \varepsilon$; $i \leftarrow 0$
for token τ in $\mathrm{LLMStream}(x, \ell, \varphi, \mathit{slots})$ **do**
  $\mathit{buf} \leftarrow \mathit{buf} \,\|\, \tau$
  if $\textsc{SentenceEnd}(\mathit{buf})$ **and** $i < m$ **then**
    $a \leftarrow \textsc{CacheGet}(\textsc{hash}(\mathit{buf}, v, \ell))$
    if $a = \emptyset$ **then** $a \leftarrow \mathrm{TTS}(\mathit{buf}, \ell)$ **end if**
    $\textsc{Emit}(a)$; $i \leftarrow i{+}1$; $\mathit{buf} \leftarrow \varepsilon$
  end if
end for
if all providers failed **then** $\textsc{Emit}(\textsc{HoldLine}(\ell))$ **end if**
$\mathit{slots} \leftarrow \textsc{Merge}(\mathit{slots}, \textbf{await}\ E)$ ▷ *Eq. (4)*
$\textsc{PersistAsync}(\mathit{transcript}, \mathit{latencies}, \mathit{slots})$

IV. Implementation and Experimental Evaluation

A. Implementation and Environment

The agent is written in Python (FastAPI, asyncio, httpx, Pydantic) on the Pipecat framework [16]; workers run on Cloudflare Workers (TypeScript); the CRM is a Next.js application; persistence is PostgreSQL 17 with 13 tables, 36 indexes, 37 tenant-isolation policies, one view, four functions and one trigger across 18 migrations. Recognition uses a commercial Indic streaming recogniser with batch fallback; generation uses Gemini 2.5 Flash with an OpenAI-compatible fallback; synthesis uses a commercial Indic TTS with three alternative provider adapters. The test suite comprises 387 Python tests, 116 worker tests and a pgTAP isolation test. During pilot sessions the agent ran on a laptop (AMD Ryzen 7 7840HS, 8 cores/16 threads, 15 GB usable RAM, Windows 11) exposed to the telephony provider through a tunnel; all model services were remote cloud APIs reached from Chennai, India.

B. Data

Per-turn telemetry was logged by the production WebSocket handler and a local harness during internal pilot sessions conducted by the development team and testers; no customer traffic is included. Of 115 sessions, 52 contained at least one completed lead turn, yielding 421 turns (Hindi 196, English 176, Tamil 49). The median session length was 98 s (p95 291 s, maximum 450 s) with a median of 6 turns. Lead-turn intents were normal 332, backchannel 32, silence 31, garbled 15, close 7, clarify 2, off-topic 1 and reject 1; the reply language changed on 18 turns.

TABLE II
TIME-TO-FIRST-AUDIO COMPONENTS (MS)

Measure	n	Mean	p50	p90	p95
$\hat{T}$, all turns	397	1688	1567	2627	3236
First LLM sentence	382	887	616	1438	1672
First TTS sentence	358	679	430	1448	1805
Batch STT	154	483	442	620	701

As Table II shows, $\hat{T}$ was below 1 s on 21.2% of turns, below 1.5 s on 47.1% and below 2 s on 77.6%. Generation contributed 50.6% of mean $\hat{T}$, synthesis 38.3% and recognition 11.1% (Fig. 2). The median $\hat{T}$ by reply language was 1482 ms (English), 1640 ms (Hindi) and 1414 ms (Tamil).

C. Ablation Analysis

TABLE III
OBSERVATIONAL ABLATIONS OF TIME TO FIRST AUDIO (MS)

Condition	n	Mean	p50	Δp50
Batch STT	154	1942	1760	–
Streaming STT	243	1527	1387	−21.2%
Synthesised first sentence	358	1762	1624	–
Cached first sentence	39	1010	997	−38.6%

Streaming recognition removes the recognition term from (2), and the batch condition additionally incurs the 750 ms silence window in Δ_{EP}, which (2) excludes; the end-to-end advantage of streaming is therefore larger than Table III shows. The comparisons are observational: the conditions co-vary with session date, code version and phrase length.

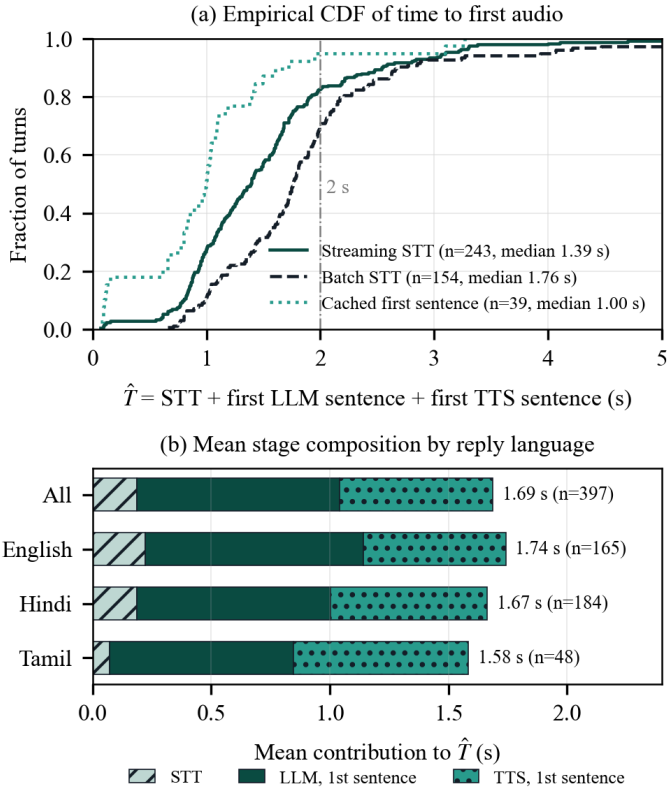


Fig. 2. Time to first audio over 397 logged turns. (a) Empirical CDF for streaming-STT, batch-STT and cached-first-sentence turns. (b) Mean contribution of recognition, first-sentence generation and first-sentence synthesis by reply language.

D. Provider Benchmark

Table IV was obtained with identical inputs through the agent's own client code from Chennai on 3 November 2026 (LLMs: one prompt, time to first non-empty token; TTS: one 81-character Hindi sentence, full synthesis time; gpt-oss models served by Groq). Providers A, B and C are ElevenLabs, Cartesia and Smallest, respectively. Four Gemini requests failed with HTTP 429 under the free-tier quota, confirming the need for the availability policy of Section III. The benchmark measures first-token and full-synthesis times, which are not directly comparable with the first-sentence quantities of Table II; it indicates the relative ordering of providers on this network, not end-to-end gains.

TABLE IV
CONTROLLED PROVIDER BENCHMARK, TEN TRIALS (MS)

Service	n	Mean	p50	Min	Max
LLM gpt-oss-20b, first token	10	566	529	447	846
LLM gpt-oss-120b, first token	10	771	789	461	1136
LLM Gemini 2.5 Flash, first token	6	1198	994	845	1990
TTS provider A, Hindi sentence	10	892	663	491	1482
TTS provider B, Hindi sentence	10	1141	1028	730	1547
TTS provider C, Hindi sentence	10	1293	1201	1026	1783

E. Database Verification

All migrations were executed on fresh PostgreSQL instances and exercised with inserts. Isolation tests passed 4/4: each tenant observed only its own lead and cross-tenant inserts were rejected. Constraint tests rejected duplicate (tenant, phone) pairs, out-of-range scores and unknown statuses. Table V lists the defects that execution exposed.

V. DISCUSSION AND LIMITATIONS

Complexity and scalability. Per-turn computation in the agent is $O(|x|)$ in transcript length (normalisation, state updates, hashing); latency is dominated by remote services, so throughput scales with concurrent asynchronous sessions until network or provider rate limits bind. Per-call state resides in process memory, so horizontal scaling requires call-affinity routing. Each call writes about 21 rows across the schema of Fig. 3; at 100 tenants × 2000 calls per month this is about 4.2×10^6 rows per month, which motivates monthly range partitioning of the event tables, transaction-mode connection pooling and pre-aggregated views. Tenant-leading composite indexes align with the isolation predicates, and the query plan of the CRM list query is an index scan.

Failure modes. (i) Generation dominates delay and its tail (p95 of 1.67 s for the first sentence), and free-tier quotas produce HTTP 429 cascades. (ii) Recognition is single-provider, so exhausted credits disable the agent. (iii) $H = 2$ delays genuine unrequested switches by one turn, and trigger lists are curated by hand. (iv) Merge and dangling holds protect slow speakers but add 0.4–0.7 s to completed turns. (v) Amplitude-based barge-in is vulnerable to echo. (vi) Narrowband recognition corrupts proper nouns, and the normaliser repairs only known entities.

Threats to validity. The data are internal pilot sessions with few speakers; no word error rate, human naturalness rating or lead-scoring accuracy against sales outcomes was measured, and the ablations are observational. Latencies were measured from a client in Chennai and depend on service regions.

VI. CONCLUSION AND FUTURE WORK

A multilingual outbound voice agent was designed, implemented and characterised. Sentence-pipelined generation, streaming recognition and phrase caching yielded a median time to first audio of 1.57 s, with 77.6% of turns under 2 s across English, Hindi and Tamil, and the language model was identified as the dominant remaining cost.

Hysteresis language control addresses the failure of utterance-level language following in code-mixed calls, and enforcing tenancy and billing in the database, verified by execution, removed three defects that review had missed. Future work comprises streaming synthesis and speculative first sentences, controlled A/B ablations, word-error-rate and naturalness evaluation with external speakers, validation of lead scores against sales outcomes, regional co-location of services, and time-partitioned storage for scale.

TABLE V
DEFECTS EXPOSED BY EXECUTION–BASED SCHEMA VERIFICATION

ID	Defect	Effect	Fix
D1	CHECK constraint capped credits at 2; trigger assigned 3	every call longer than 300 s rejected	constraint widened to 0–4
D2	cost computed as credits × average order value (INR 200,000)	daily INR 600 cap exceeded after one call; dispatch halted	INR 12 per credit fallback; legacy rows repaired
D3	login-hook role blocked by row-level security on users	tokens lacked tenant claim; all tenant queries empty	role-scoped grant and policy

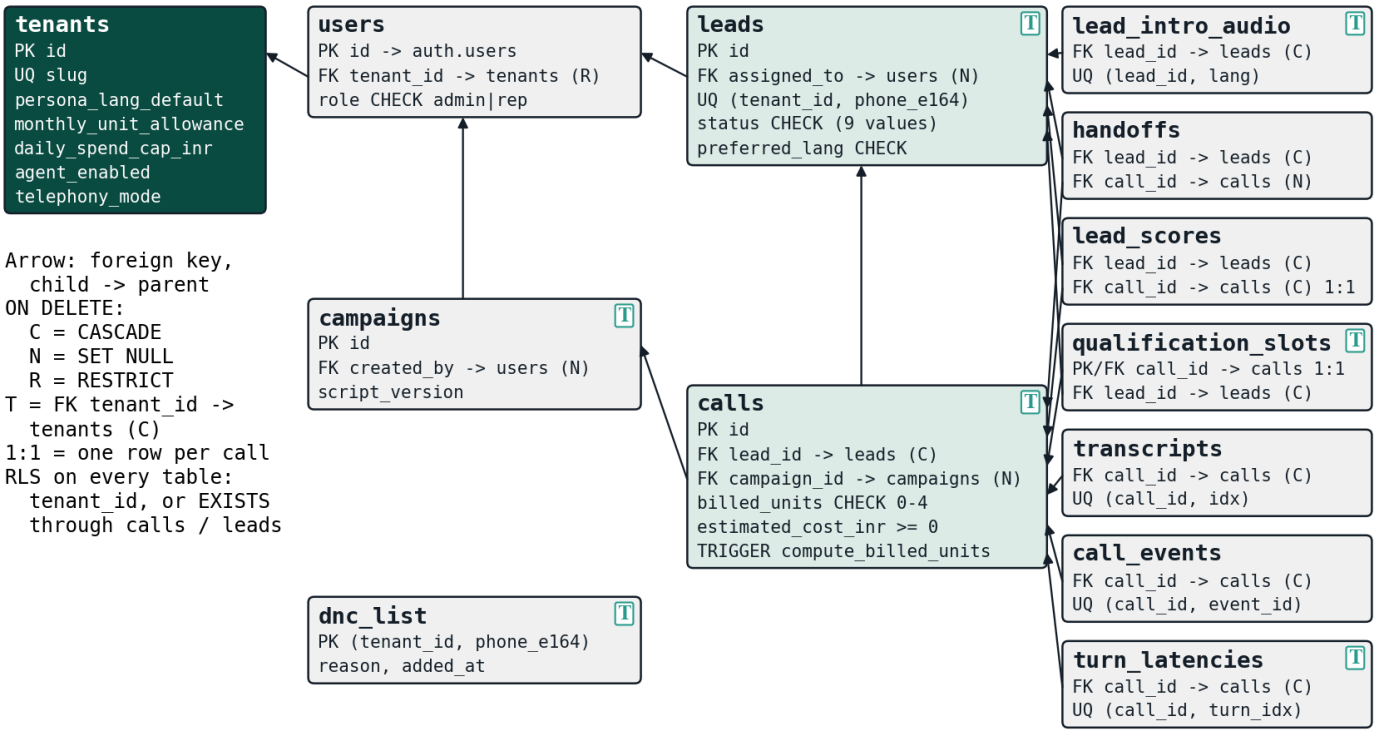


Fig. 3. Thirteen-table multi-tenant schema. Arrows are foreign keys from child to parent, annotated with their `ON DELETE` action; tables marked T also reference `tenants`. Every table is protected by row-level security, directly on `tenant_id` or through an `EXISTS` sub-query on `calls` or `leads`.

REFERENCES

[1] T. Stivers, N. J. Enfield, P. Brown, C. Englert, M. Hayashi, T. Heinemann, G. Hoymann, F. Rossano, J. P. de Ruiter, K.-E. Yoon, and S. C. Levinson, "Universals and cultural variation in turn-taking in conversation," *Proc. Natl. Acad. Sci.*, vol. 106, no. 26, pp. 10587–10592, 2009.

[2] G. Skantze, "Turn-taking in conversational systems and human-robot interaction: A review," *Comput. Speech Lang.*, vol. 67, p. 101178, 2021.

[3] M. Shannon, G. Simko, S.-Y. Chang, and C. Parada, "Improved end-of-query detection for streaming speech recognition," in *Proc. Interspeech*, 2017, pp. 1909–1913.

[4] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 28492–28518.

[5] V. Pratap *et al.*, "Scaling speech technology to 1,000+ languages," *J. Mach. Learn. Res.*, vol. 25, no. 97, pp. 1–52, 2024.

[6] T. Javed *et al.*, "IndicSUPERB: A speech processing universal performance benchmark for Indian languages," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 12942–12950.

[7] S. Sitaram, K. R. Chandu, S. K. Rallabandi, and A. W. Black, "A survey of code-switched speech and language processing," arXiv:1904.00784, 2019.

[8] S. Aulbach, T. Grust, D. Jacobs, A. Kemper, and J. Rittinger, "Multi-tenant databases for software as a service: Schema-mapping techniques," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2008, pp. 1195–1206.

[9] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[10] G. K. Kumar, P. S. V, P. Kumar, M. M. Khapra, and K. Nandakumar, "Towards building text-to-speech systems for the next billion users," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2023.

[11] J. Kim, J. Kong, and J. Son, “Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech,” in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 5530–5540.

[12] Gemini Team, “Gemini: A family of highly capable multimodal models,” arXiv:2312.11805, 2023.

[13] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Comput. Surv.*, vol. 55, no. 12, pp. 248:1–248:38, 2023.

[14] R. Nygård and J. Mezei, “Automating lead scoring with machine learning: An experimental study,” in *Proc. 53rd Hawaii Int. Conf. Syst. Sci. (HICSS)*, 2020.

[15] The PostgreSQL Global Development Group, “Row security policies, PostgreSQL 17 documentation.” [Online]. Available: https://www.postgresql.org/docs/17/ddl-rowsecurity.html

[16] Daily, “Pipecat: Open-source framework for voice and multimodal conversational AI,” GitHub repository. [Online]. Available: https://github.com/pipecat-ai/pipecat